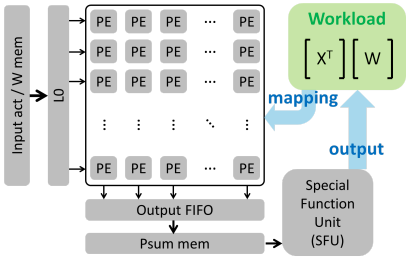
ECE 284 Final Project Report

Team: PSH
Member: Wen-Chieh Lo, Jongmin Lee, Jaewon Lee, Jing-Hua Chang, Ming-Yang Wu

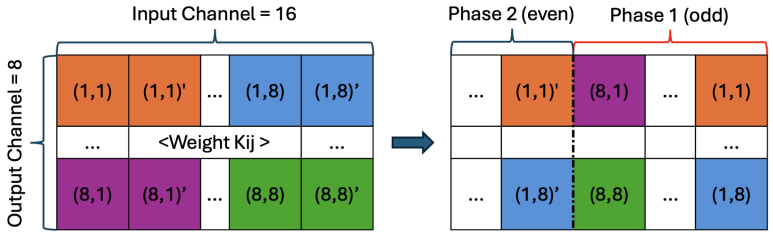
Part1

In the vanilla version, we implemented a complete 8×8 weight-stationary 2D systolic array for 4-bit VGG convolution. We designed the MAC array, integrated L0 for weight/activation streaming, built the output FIFO and psum SRAM path, and connected the Special Function Unit (SFU) for accumulation and ReLU. We verified the full dataflow—including loading weights and activations into SRAM, kernel broadcast through L0, PE execution, psum collection, accumulation, and ReLU—and confirmed correctness using the generated stimulus and expected output vectors. This establishes the baseline mapping of the squeezed VGG layer onto the systolic array and provides the foundation on which later optimizations (alphas) are applied.



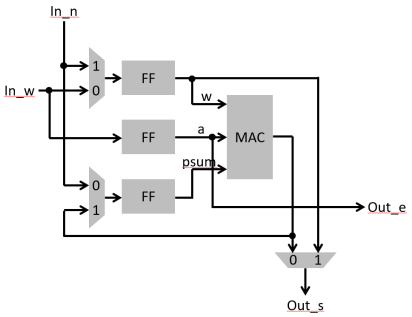
Part2

For 2bit/4bit Lane Reconfigurable SIMD Systolic Array, we will reuse many of the existing structure of vanilla, with the difference being, the array's ability to accommodate 16 input channel * 8 output channel for 2 bit activations. To load two weights per tile, we split the two consecutive weights into an odd group and even group. As shown in the figure below, we can feed weights in a two-phase fashion.



The benefit of this technique is that all of the weight loading mechanisms of vanilla can be reused without modification. The only difference is that we signal to do the loading twice. To distinguish each phase, we introduce a "phase control bit" inside of each mac tile which flips after receiving its corresponding data. Activation can be fed into the tiles just like vanilla because, although the quantity of activation has doubled, bits of each activations have halved thus maintaining its original dimensions. Each mac tile takes one 4bit activation and splits it into two 2 bit activations. Then, it multiplies it to each of the two weights and sums them up creating a psum for each tile. As each mac tile produces only one psum, everything after this can reuse vanilla design. As for 4bit, mode, we load the same weights twice and use shifting to get the same result for each tile.

Part3



The figure above shows a portion of the MAC tile designed to support both weight-stationary and output-stationary modes. Verilog implementation just follows this structure. When the MUX select signal is 0, it operates in weight-stationary mode, and when it is 1, it switches to output-stationary mode. Additionally, we added an instruction called “flush”, which outputs the completed results stored in each PE row by row once the output-stationary computation is finished. We also placed the input FIFO on the north side so that weights can propagate properly in output-stationary mode.

Compared to part 1, the number of bits in the inst port at the core module has increased, and an additional input port has been added to feed the weight memory connected to the iFIFO. Therefore, for weight-stationary mode, the original testbench from part 1 (core_tb_p1.v) still works as long as the newly added input bits of the core module are all set to zero. A separate testbench was created for part 3, and its file name is core_tb_p3.v. The text file inputs for this testbench are exactly the same as those used in part 1. However, since the activation and weight sequences required for weight-stationary mode differ from those required for output-stationary mode, core_tb_p3.v includes tasks named generate_modified_acti and transpose_weight that regenerate new text files suitable for part 3. After that, appropriate control signals are issued according to the output-stationary architecture to complete the computation.

```
[jol156@ieng6-ece-19]:part_3:1032$ vim filelist
[jol156@ieng6-ece-19]:part_3:1033$ vim core_tb_p3.v
[jol156@ieng6-ece-19]:part_3:1034$ iveri filelist
[jol156@ieng6-ece-19]:part_3:1035$ irun
VCD info: dumpfile core_tb_p3.vcd opened for output.
Part 3: Output stationary
 0-th output featuremap Data matched! :D
 1-th output featuremap Data matched! :D
 2-th output featuremap Data matched! :D
 3-th output featuremap Data matched! :D
 4-th output featuremap Data matched! :D
 5-th output featuremap Data matched! :D
 6-th output featuremap Data matched! :D
 7-th output featuremap Data matched! :D
############ No error detected ##############
########### Project Completed !! ############
[jol156@ieng6-ece-19]:part_3:1036$ vim filelist
[jol156@ieng6-ece-19]:part_3:1037$ iveri filelist
[jol156@ieng6-ece-19]:part_3:1038$ irun
VCD info: dumpfile core_tb_p1.vcd opened for output.
Part 1: test
############ Verification Start during accumulation ############
 1-th output featuremap Data matched! :D
 2-th output featuremap Data matched! :D
 3-th output featuremap Data matched! :D
 4-th output featuremap Data matched! :D
 5-th output featuremap Data matched! :D
 6-th output featuremap Data matched! :D
 7-th output featuremap Data matched! :D
 8-th output featuremap Data matched! :D
 9-th output featuremap Data matched! :D
10-th output featuremap Data matched! :D
11-th output featuremap Data matched! :D
12-th output featuremap Data matched! :D
13-th output featuremap Data matched! :D
14-th output featuremap Data matched! :D
15-th output featuremap Data matched! :D
16-th output featuremap Data matched! :D
############ No error detected ##############
########### Project Completed !! ############
[jol156@ieng6-ece-19]:part_3:1039$
```

As shown in the figure right, the same hardware correctly supports both the weight-stationary mode (part 1) and the output-stationary mode (part 3)

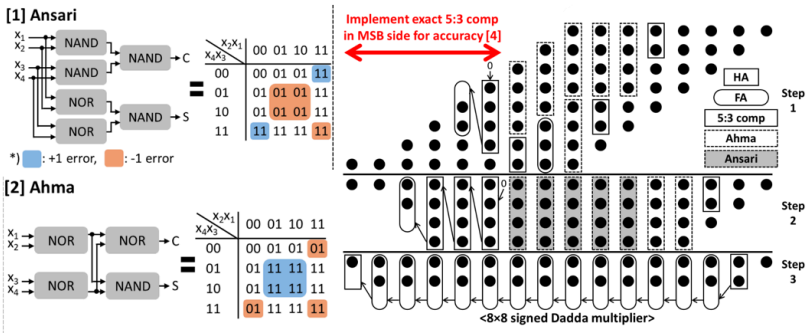
Part4 - alpha1

In homework 7, we have examined 2 different pruning strategies for quantized VGGNet models. Our results have shown that unstructured pruning generally achieves higher re-train accuracy compared to structured pruning, whereas structured pruning offers superior regularity in the resulting weight masks. This regularity is advantageous for sparse-aware systolic arrays, as it improves hardware mapping efficiency. Therfore, we decided to merge the benefits of both structured and unstructured pruning to build a model with better retrain accuracy while retaining regularity.

	Structured	Hybrid	Unstructured
After Pruning	10%	10%	10%
Retrain	73%	87%	90.00%

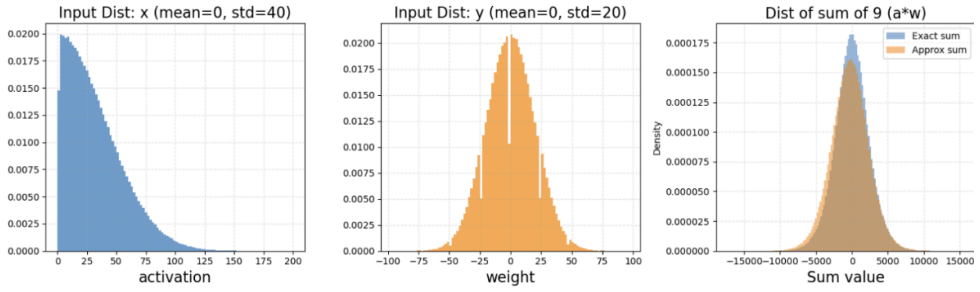
The sparsity for all 3 pruning strategies are set to 80%. For hybrid pruning, we set structured pruning sparsity to 50% and combine with extra 60% of unstructured pruning to reach a final 80% of sparsity.

Part4 - alpha2

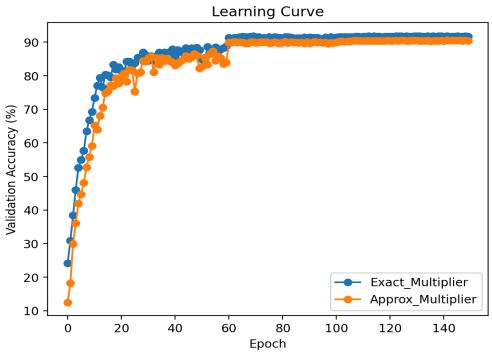


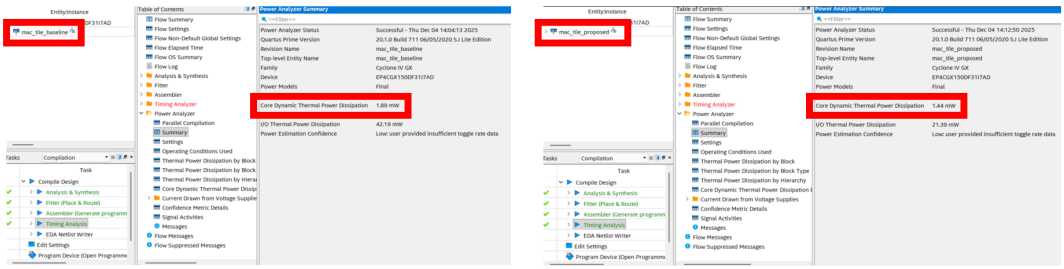
A 4:2 compressor takes four 1-bit inputs, sums them, and expresses the result using only 2 bits. No matter how well a compressor is designed, when all four inputs are 1'b1, the sum becomes 4 (i.e., 3'b100), and representing this value with only 2 bits inevitably introduces error. In Alpha 2, the compressor is composed of simple NOR and NAND gates without any XOR or XNOR gates (references [1], [2]).

Since the error distributions of these compressors are quite symmetry, combining those two allows the errors to compensate for each other during partial sum reduction (reference [3]). This approach achieves high accuracy for unsigned integer operations but for signed integer operations, accuracy degrades due to the error sensitivity around the MSB region. To address this issue, in this work we additionally incorporated the idea from reference [4]. Only the upper MSB part is implemented using an exact 5:3 compressor instead of an approximate 4:2 compressor. This increases accuracy with minimal hardware overhead. As a result, we obtained an approximate multiplier that achieves the following error metrics without XOR, XNOR based 4:2 compressor: NMED (Normalized Mean Error Distance): 9.510e-3, MRED (Mean Relative Error Distance): 0.1584, MRERR (Mean Relative Error): 0.01103, PRED2 (probability of RED > 2%): 0.6103, PRED50: 0.04071.



Assuming Gaussian input distributions, we compared the output distribution of nine consecutive MAC operations and found that the results from the approximate multiplier closely match those of the exact multiplier.





<Baseline = accurate multiplier>

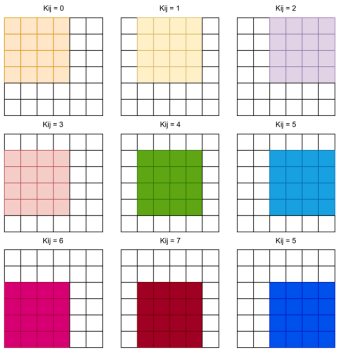
<Proposed = approximate multiplier>

Furthermore, synthesis results for the MAC tile show that power consumption was reduced from **1.89 mW to 1.44 mW, achieving a 23.81% reduction.**

Aside from power efficiency, we achieve this without sacrificing much accuracy on the training front – **approximate multiplier attains 90.61% accuracy, which is not far from the accurate multiplier's 91.82%** (given all training parameters fixed). We implemented this by replacing the traditional F.conv2d function in the forward part with a self-defined convolution function, which contains an approximate multiplier, and then trained the model. What's worth noting is that the learning curve of the approximate multiplier is quite immune to fluctuations, unlike many approximation methods which are prone to instability during training (results from the compounding errors through iteration). This shows that the approximate multiplier is stable enough through training, and can be reliably implemented onto the training model.

Part4 - alpha3

In standard systolic-array convolution, each position loads a full receptive field even when only part of the input is actually needed, causing unnecessary activation transfers and extra computation. To remove this overhead, we introduce a Region-Aware Activation Extraction mechanism. For every convolution index K_{ij}, we detect the exact valid portion of the input feature map that contributes to the output. Instead of blindly extracting all 6 x 6 activations for every kernel position, our method dynamically identifies only the valid window for each Kernel and forwards those activations to the compute array.



For a **6 × 6 region** with a 3 × 3 kernel, the naïve method performs **36 × 9=324 activation accesses**. Region-aware extraction reduces this to **16 × 9 = 144**, cutting nearly half of the operations. Overall, this

optimization saves **44%** of the total activation-processing workload and contributes to a **26%** improvement in total execution time.

Part4 - alpha4

Traditional systolic-array dataflows serialize activation and weight transfers, forcing both to share a single read path and creating avoidable stalls. To provide more flexibility in how data is delivered into the array, we introduce a **Hybrid Read Mode** controlled by the L0 read signal. The design supports two modes:

1. **Propagate**: The read signal advances to the next row **one cycle at a time**, propagating sequentially down the array.
2. **Simultaneous**: The read signal is broadcast to **all rows in the array at once**, enabling concurrent activation of every row.

In addition, we redesign the FIFO structure to support **pipelined read/write operations**, allowing the system to load data into the FIFO while writing out data at the same time. This significantly reduces FIFO pressure, lowering the required FIFO depth from **64 entries to 16** while maintaining stable throughput. The improved concurrency reduces stalls during data movement and shortens the overall convolution schedule. In practice, this optimization leads to a **32% reduction in total cycles.**

Part4 - alpha5

Because many activations and weights are zero in CNN layers, a large number of MAC operations do not affect the final output. To avoid wasting switching activity, we introduced zero-skipping logic. We tested 2 different gating mechanism covered in lectures this quarter:

1. Gating a/b/c flip-flops → no power savings.
2. Gating MAC inputs → multiplier and adder are disabled when zero appears.

The input-gating method significantly lowers MAC switching activity and results in **23.7% total power reduction**, while flip-flop gating offers no measurable benefit. This confirms that meaningful savings occur only when the actual MAC datapath is disabled during zero cycles.

	Max Frequency	Dynamic Power	Total Logic Elements	TOPs	TOPs/W
No zero-skipping	130.57 MHz	28.53 mW	13,098	0.0167	0.5853
Zero-skipping	109.70 MHz	21.79 mW	13,831	0.0140	0.6444